

Laboratory Record 4

Course :	Full Stack Web Development
Course Code:	23CM4121
Experiment no.:	4
Student Name:	R. Srivalli
Roll No.:	A24126552108

1. TITLE

Develop Dynamic Webpage Using JavaScript

2. AIM

To develop a dynamic webpage using JavaScript that accepts student details and dynamically displays the entered name and course using DOM manipulation and event handling..

3. OBJECTIVE

- To understand the concept of **dynamic webpages using JavaScript**.
 - To learn how to select and access HTML elements using the **DOM**.
 - To handle user actions using **event listeners**.
 - To read and validate values entered by the user.
 - To dynamically create HTML elements using `createElement()`.
 - To modify and display webpage content using `textContent`, `innerHTML`, and `appendChild()`.
 - To understand how JavaScript can dynamically update webpage content without reloading the page..
-

4. THEORY

A dynamic webpage changes its content based on user actions. JavaScript makes this possible by interacting with the DOM (Document Object Model).

In this program, JavaScript selects the input fields and button using `getElementById()`. An event listener detects the button click, reads and validates the entered name and course, and then dynamically creates a profile card using `createElement()`.

The methods `textContent`, `classList.add()`, and `appendChild()` are used to add content, styling, and elements to the webpage without reloading it. Thus, the program demonstrates DOM manipulation, event handling, input validation, and dynamic content creation.

5. ALGORITHM / PROCEDURE

1. Create the HTML webpage with input fields for **name** and **course**.
 2. Select the required HTML elements using `getElementById()`.
 3. Add a **click event listener** to the Display Details button.
 4. Read and trim the values entered by the user.
 5. Check whether both input fields are filled.
 6. Display an error message if any field is empty.
 7. Clear the previous output.
 8. Dynamically create a profile card using `createElement()`.
 9. Create heading and paragraph elements for the student's name and course.
 10. Add the created elements to the profile card using `appendChild()`.
 11. Display the profile card inside the output section.
 12. Execute the program and verify the dynamic output.
-

6. PROGRAM

A : index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>Dynamic Webpage using JavaScript</title>

  <link rel="stylesheet" href="stylerec4.css">
</head>

<body>

  <div class="container">

    <header>
      <h1>Dynamic Webpage</h1>
      <p>Developed using JavaScript DOM Manipulation</p>
    </header>

    <main>
```

```
<section class="input-section">

    <h2>Enter Your Details</h2>

    <input
      type="text"
      id="nameInput"
      placeholder="Enter your name"
    >

    <input
      type="text"
      id="courseInput"
      placeholder="Enter your course"
    >

    <button id="displayBtn">
      Display Details
    </button>

</section>

<section class="output-section">

    <h2>Dynamic Output</h2>

    <div id="output">
      <p>Enter your details and click the button.</p>
    </div>

</section>

</main>

<footer>
  <p>Experiment 4 | Dynamic Webpage using JavaScript</p>
</footer>

</div>

<script src="scriptrec4.js"></script>

</body>
</html>
```

B : style.css

```
/* Experiment 4: Dynamic Webpage using JavaScript */

/* Basic page styling */
```

```
* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

body {
  font-family: Arial, sans-serif;
  background: #eef2f7;
  color: #222;
  min-height: 100vh;
}

/* Main container */
.container {
  width: 90%;
  max-width: 850px;
  margin: 40px auto;
  background: white;
  border-radius: 12px;
  overflow: hidden;
  box-shadow: 0 5px 20px rgba(0, 0, 0, 0.12);
}

/* Header */
header {
  background: #263238;
  color: white;
  text-align: center;
  padding: 30px 20px;
}

header h1 {
  margin-bottom: 8px;
}

/* Main content */
main {
  padding: 30px;
}

section {
  margin-bottom: 25px;
}

h2 {
  margin-bottom: 18px;
  color: #263238;
}

/* Input area */
```

```
.input-section {
  padding: 25px;
  background: #f5f7f9;
  border-radius: 10px;
}

input {
  width: 100%;
  padding: 12px;
  margin-bottom: 12px;
  border: 1px solid #b0bec5;
  border-radius: 6px;
  font-size: 15px;
}

input:focus {
  outline: none;
  border-color: #455a64;
}

/* Button */
button {
  padding: 12px 22px;
  background: #37474f;
  color: white;
  border: none;
  border-radius: 6px;
  cursor: pointer;
  font-size: 15px;
}

button:hover {
  background: #263238;
}

/* Dynamic output */
.output-section {
  padding: 25px;
  border: 1px solid #d5dce0;
  border-radius: 10px;
}

#output {
  padding: 18px;
  background: #fafafa;
  border-left: 5px solid #546e7a;
  min-height: 70px;
}

.profile-card {
  background: #eceff1;
}
```

```

padding: 18px;
border-radius: 8px;
}

.profile-card p {
margin: 8px 0;
}

.profile-card strong {
color: #263238;
}

/* Footer */
footer {
text-align: center;
background: #eceff1;
padding: 15px;
color: #455a64;
font-size: 14px;
}

/* Responsive design */
@media (max-width: 600px) {

.container {
width: 94%;
margin: 20px auto;
}

main {
padding: 20px;
}

header {
padding: 25px 15px;
}

.input-section,
.output-section {
padding: 20px;
}
}

```

C : script.js

```

// Experiment 4: Dynamic Webpage using JavaScript
// Demonstrates DOM manipulation and event handling

// Selecting HTML elements using their IDs
const nameInput = document.getElementById("nameInput");

```

```
const courseInput = document.getElementById("courseInput");
const displayBtn = document.getElementById("displayBtn");
const output = document.getElementById("output");

// Event listener for the button
displayBtn.addEventListener("click", function () {

    // Read values entered by the user
    const name = nameInput.value.trim();
    const course = courseInput.value.trim();

    // Check whether the input fields are empty
    if (name === "" || course === "") {

        output.textContent = "Please enter both name and course.";

        return;
    }

    // Clear the previous output
    output.innerHTML = "";

    // Dynamically create a new div element
    const profileCard = document.createElement("div");

    // Add CSS class dynamically
    profileCard.classList.add("profile-card");

    // Create heading dynamically
    const heading = document.createElement("h3");
    heading.textContent = "Student Details";

    // Create paragraph for name
    const namePara = document.createElement("p");
    namePara.innerHTML = "<strong>Name:</strong> " + name;

    // Create paragraph for course
    const coursePara = document.createElement("p");
    coursePara.innerHTML = "<strong>Course:</strong> " + course;

    // Add elements to the dynamically created card
    profileCard.appendChild(heading);
    profileCard.appendChild(namePara);
    profileCard.appendChild(coursePara);

    // Display the dynamically created content
    output.appendChild(profileCard);

});
```

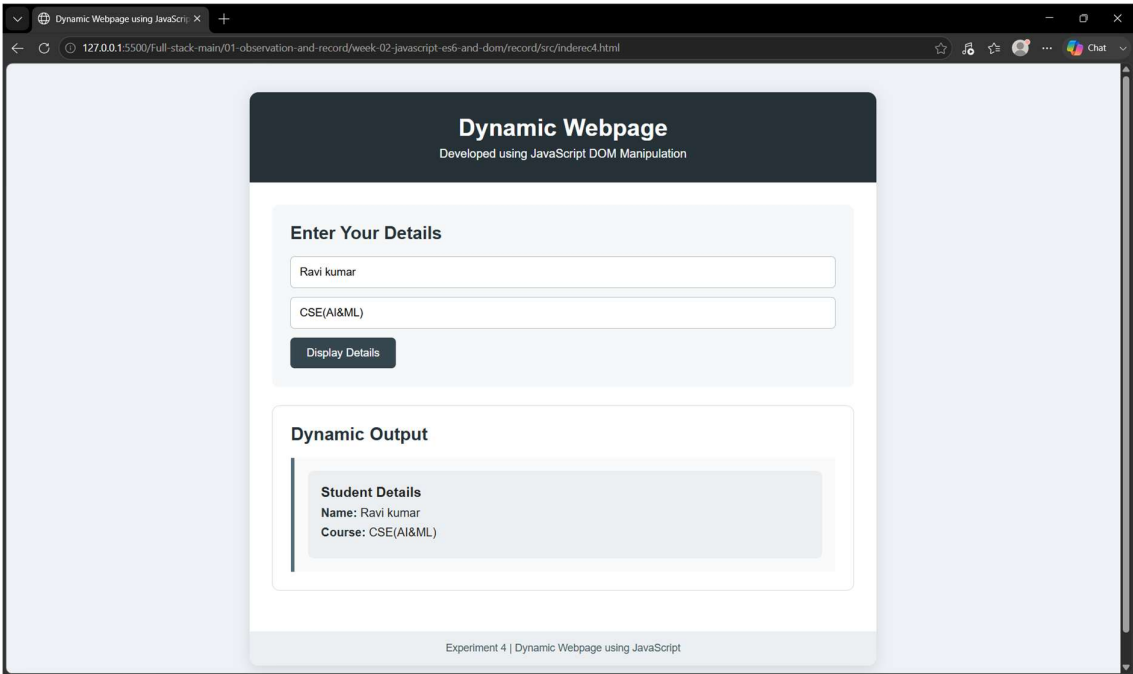
7. CODE EXPLANATION

S.No.	Code / Component	Explanation
1	getElementById()	Selects HTML elements such as input fields, button, and output area.
2	nameInput	References the input field used to enter the student's name.
3	courseInput	References the input field used to enter the student's course.
4	displayBtn	References the Display Details button.
5	output	References the section where dynamic content is displayed.
6	addEventListener("click", ...)	Detects when the Display Details button is clicked.
7	.value	Retrieves the values entered in the input fields.
8	.trim()	Removes unnecessary spaces from the beginning and end of the input.
9	`if (name === ""	
10	output.textContent	Displays a message when the required details are not entered.
11	output.innerHTML = ""	Clears the previous dynamic output.
12	document.createElement()	Dynamically creates new HTML elements using JavaScript.
13	classList.add()	Adds the CSS class profile-card to the newly created element.
14	textContent	Sets the text content of the dynamically created heading.
15	innerHTML	Adds formatted content such as Name: and Course: to the paragraphs.
16	appendChild()	Adds the dynamically created elements to the profile card and output section.
17	profileCard	Represents the dynamically created student details card.
18	return	Stops further execution when the input validation fails.

8. TEST CASES AND EXECUTION DATA

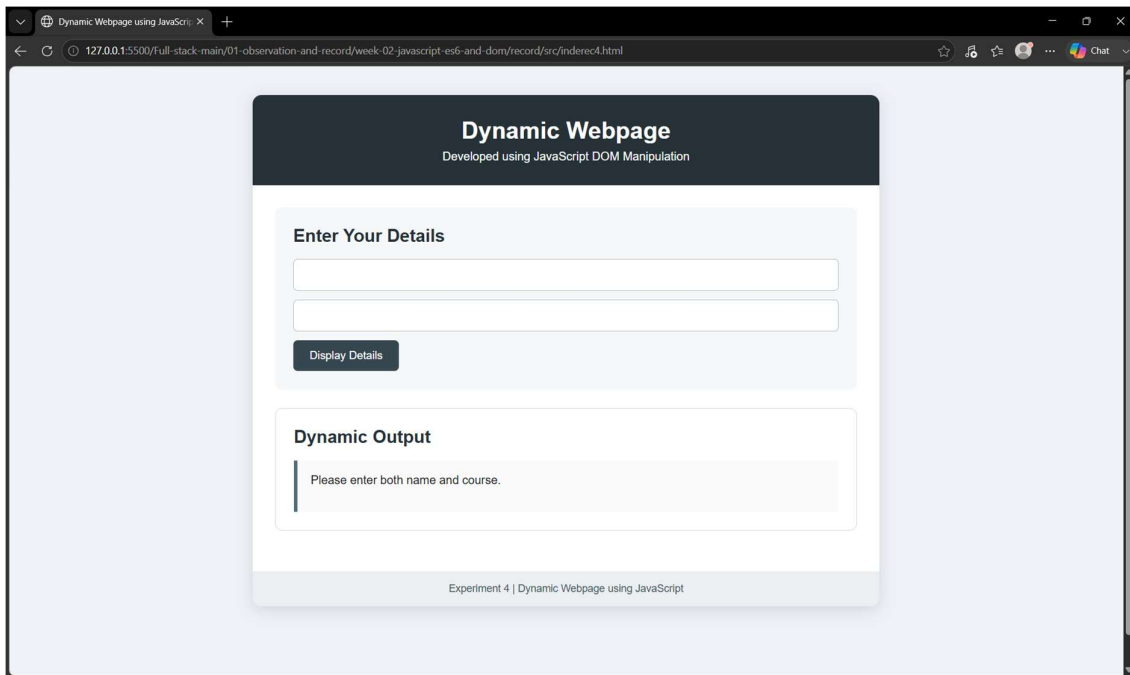
Test Case 1 – Valid Student Details

Input / Action	Expected Result	Status
Enter Name: Ravi Kumar and Course: CSE (AI & ML), then click Display Details	Student details are dynamically displayed in the output section	Pass



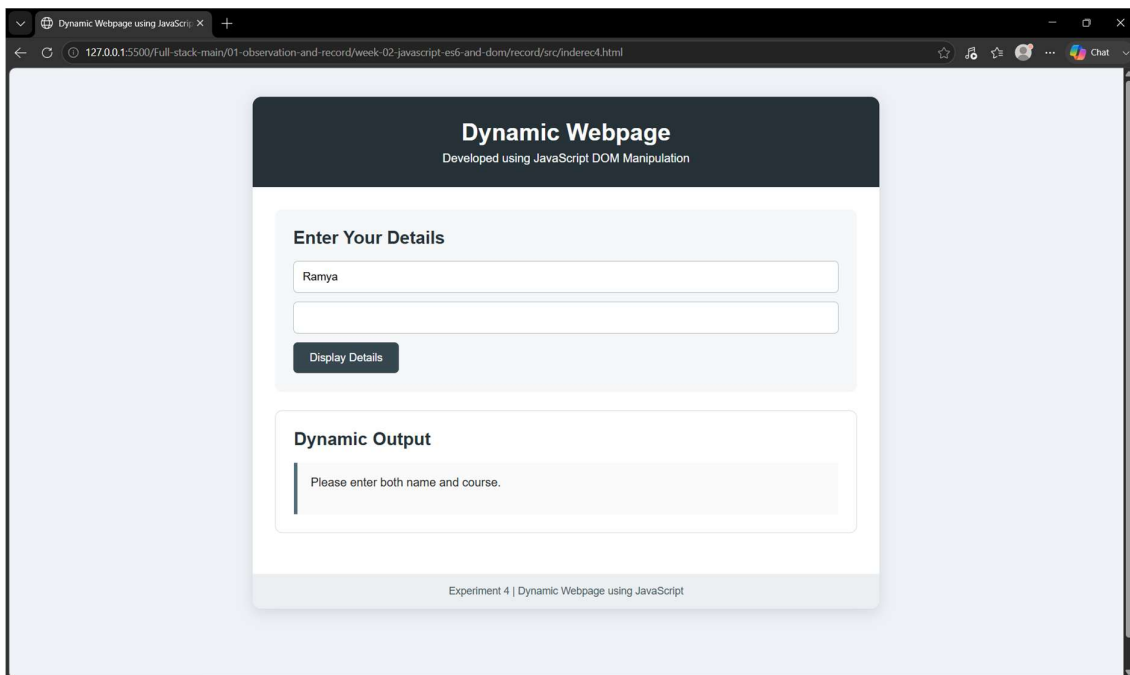
Test Case 2 – Invalid/Whitespace Input

Input / Action	Expected Result	Status
Enter only spaces in the Name or Course field and click Display Details	Program treats the field as empty and displays the validation message	Pass



Test Case 3 – Empty Course Field

Input / Action	Expected Result	Status
Enter a name but leave the Course field empty, then click Display Details	Message “Please enter both name and course.” is displayed	Pass



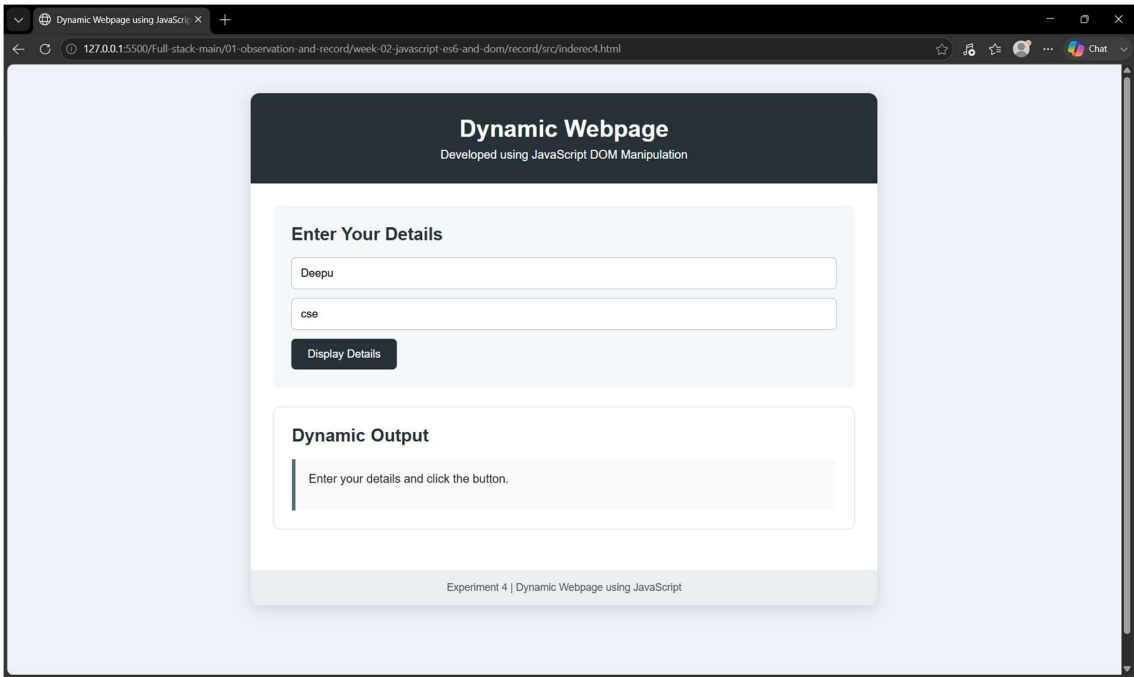
Test Case 10 – Missing JavaScript File

Input / Action	Expected Result	Status
Open the HTML file when scriptrec4.js is missing or incorrectly linked	Display button will not generate the profile	Fail

```

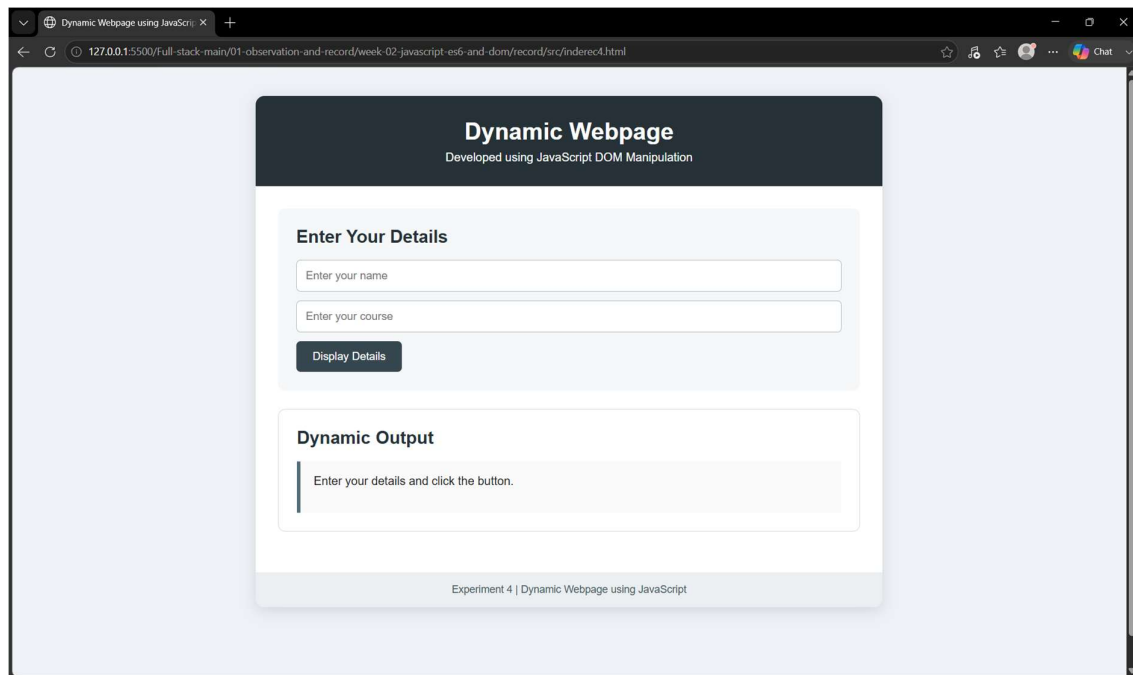
    <footer>
      <p>Experiment 4 | Dynamic Webpage using JavaScript</p>
    </footer>
  </div>
  <script src="scriptrec.js"></script>
</body>
</html>
```

Display button doesn't work

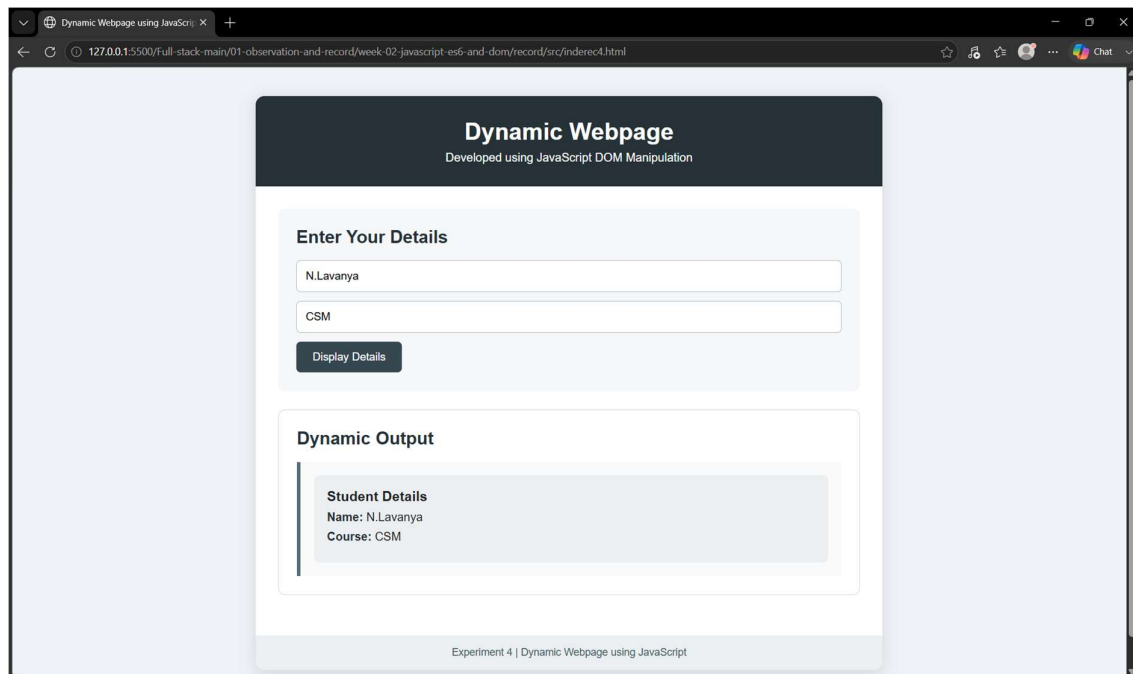


9. OUTPUT

Output 1 – Initial Webpage



Output 2 – Dynamic Student Details



10. RESULT

The Dynamic Webpage using JavaScript was successfully developed using DOM manipulation and event handling. The webpage accepts student details, validates the input, and dynamically creates and displays the student profile without reloading the page.